

# **Technical Report**

## **ChatAI Console Application**

*A C# Conversational AI System with OpenAI API Integration*

### **Prepared by:**

Isaiah Alfred

Alesha McRae

David Jean-Charles Bahidika

### **Submitted to:**

Dr. Sumarga K. Sah Tyagi

Intro to C# Programming

Florida A&M University

April 30<sup>th</sup>, 2026

*Source Code: [github.com/davidbah2703/ChatAi](https://github.com/davidbah2703/ChatAi)*

## **Abstract**

ChatAI is a C# console-based application that integrates the OpenAI GPT API to deliver real-time, context-aware conversational AI. Built on the .NET Generic Host framework, the system applies a layered architecture separating Models, Services, UI, and Testing concerns. Key engineering decisions include interface-driven dependency injection, async/await API communication, Polly-based exponential backoff retry logic, Serilog structured logging with rolling file output, and JSON-based conversation persistence. This report provides a detailed technical analysis of the system's architecture, component design, implementation decisions, data flow, and testing strategy.

## **1. Introduction**

Modern conversational AI applications require more than a direct connection to a language model API. They require careful architectural design to ensure that interactions remain context-aware, the system stays responsive under network variability, and data is preserved reliably across sessions. ChatAI was built to address all of these concerns within the constraints of a C# console application.

The project was developed as a team effort by Isaiah Alfred, Alesha McRae, and David Bahidika for an Intro to C# Programming course at Florida A&M University. The goal was not simply to wrap the OpenAI API in a loop, but to apply professional-grade software engineering practices, including layered architecture, interface-based dependency injection, asynchronous programming, resilience patterns, and unit testing, to produce a system that behaves predictably and is maintainable over time.

This report documents every significant technical decision made during development, grounded in the actual source code hosted at [github.com/davidbah2703/ChatAi](https://github.com/davidbah2703/ChatAi). Sections cover the system's architecture, each component's design and responsibilities, the data flow across a full conversation cycle, implementation details, testing strategy, and future improvement opportunities.

## **2. System Overview**

ChatAI enables users to hold ongoing AI-powered conversations through a console interface. Unlike stateless API wrappers that treat each message independently, ChatAI maintains full conversation history across the session, passes recent context to every API request, analyzes the sentiment of each user message in real time, and supports saving and reloading entire conversations as JSON files.

## **2.1 Project Structure**

The repository is organized into the following top-level directories and files:

| Path             | Description                                                                              |
|------------------|------------------------------------------------------------------------------------------|
| Models/          | Core data structures: Message.cs and Conversation.cs                                     |
| Services/        | Business logic: ConversationManager, OpenAIService, SentimentService, PersistenceService |
| UI/              | Console interface: ConsoleUI.cs, user input, command dispatch, output display            |
| Tests/           | xUnit unit tests covering message handling, sentiment, and persistence                   |
| Program.cs       | Application entry point, configures .NET Generic Host, DI container, Serilog             |
| ChatAI.csproj    | Project file, .NET 6 target, NuGet dependencies (Polly, Serilog, Microsoft.Extensions.*) |
| appsettings.json | Configuration file, OpenAI API key section, log level settings                           |

## **3. Architecture Design**

ChatAI follows a layered architecture pattern that strictly separates data structures, business logic, user interaction, and test validation into independent layers. This design prevents any single component from carrying too many responsibilities and makes the system easy to extend or modify without cascading changes across the codebase.

### **3.1 Layer Overview**

| Layer          | Responsibility                                                                           |
|----------------|------------------------------------------------------------------------------------------|
| Models         | Define shared data structures used by all other layers                                   |
| Services       | Implement all business logic: API calls, conversation management, sentiment, persistence |
| UI (ConsoleUI) | Handle user input/output, dispatch commands, coordinate services                         |
| Tests          | Validate service behavior in isolation using unit tests                                  |

Each layer depends only on the layer below it. The UI layer calls into Services; Services operate on Models; no layer has visibility into layers above it. This unidirectional dependency graph means that a change to the persistence logic, for example, cannot break the UI layer unless the service interface itself changes.

### **3.2 Interface-Based Design**

Every service in ChatAI is defined through a C# interface and registered against that interface in the DI container. The actual types registered are:

- ISentimentService → SentimentService
- IConversationManager → ConversationManager
- IOpenAIService → OpenAIService (registered via AddHttpClient<IOpenAIService, OpenAIService>())

This means no class in the system holds a direct reference to a concrete implementation. The ConsoleUI holds an IOpenAIService and an IConversationManager, it has no knowledge of how either one works internally. This design makes every component independently mockable for testing and independently swappable without touching the classes that depend on it.

## **4. Component Design**

### **4.1 Models Layer**

The Models layer defines the two data structures that flow through the entire application. By centralizing these definitions, all layers work with the same types and there is no ambiguity in how data is shaped or interpreted.

#### **Message.cs**

Represents a single unit of communication. Each Message stores:

- Content, the raw text of the message
- Role, whether the message came from the user or the AI assistant
- Timestamp, the exact date and time the message was created
- Sentiment, a classification of Positive, Negative, or Neutral assigned by the SentimentService at the time of storage

#### **Conversation.cs**

Represents the full session as an ordered list of Message objects. The ConversationManager reads from and writes to this structure for the duration of the session, and the PersistenceService serializes and deserializes it for save and load operations.

## **4.2 Services Layer**

### **ConversationManager.cs**

The ConversationManager is the central data service of the application. All message storage and retrieval flows through it, which ensures the conversation history is never written to directly by the UI or the API service. Its responsibilities include:

- Appending every user and AI message to the active Conversation object as it is created
- Returning the last N messages (configurable, default ten) to provide rolling context for each API request
- Exposing a keyword search method that scans message content across the full history
- Computing and returning sentiment distribution statistics across all stored messages

The ConversationManager is registered as a singleton, meaning the same instance is used for the entire lifetime of the application. This guarantees that all components reading from or writing to the history are operating on the same data.

### **OpenAIService.cs**

The OpenAIService handles all outbound communication with the OpenAI API. It is registered using `AddHttpClient<IOpenAIService, OpenAIService>()`, which gives it access to a managed `HttpClient` instance with proper connection pooling and lifetime management. For each request, the service:

1. Receives the current user message from ConsoleUI
2. Retrieves recent conversation history from IConversationManager
3. Builds a request payload containing the system prompt, history context, and current message

4. Sends the request asynchronously and awaits the response
5. Returns the generated response string to ConsoleUI for display and storage

All API calls are made with `async/await` to keep the application non-blocking. Resilience is handled by Polly, which wraps each request in an exponential backoff retry policy. If the request fails due to a transient network error or rate limit, Polly waits an increasing interval (e.g., 2s, 4s, 8s) before retrying, up to a configured maximum number of attempts.

### **SentimentService.cs**

The `SentimentService` evaluates the text content of each user message and returns a sentiment classification of Positive, Negative, or Neutral. The classification is attached to the `Message` object before it is stored in the `ConversationManager`. This data is used by the `stats` command to produce a distribution summary across the full conversation history.

### **PersistenceService.cs**

The `PersistenceService` serializes and deserializes the active `Conversation` object using `System.Text.Json`. When the user runs the `save` command, the full conversation, including all messages, roles, timestamps, and sentiment labels, is written to a JSON file on disk. When the user runs `load`, that file is deserialized back into a `Conversation` object and passed to the `ConversationManager` to replace the current session.



### **4.3 UI Layer, ConsoleUI.cs**

The ConsoleUI is the application's entry point for user interaction. It holds references to IOpenAIService and IConversationManager and coordinates all activity between them. On each iteration of its main loop, ConsoleUI:

6. Reads a line of input from the console
7. Checks whether the input matches any registered command keyword
8. If it is a command, routes it to the appropriate handler
9. If it is a message, passes it to IConversationManager for storage and sentiment analysis, then calls IOpenAIService.SendAsync() with the message and context
10. Receives the AI response, stores it via IConversationManager, and prints it to the console

The full list of supported commands is:

| Command          | Behavior                                                                  |
|------------------|---------------------------------------------------------------------------|
| help             | Prints all available commands and their descriptions                      |
| exit             | Flushes Serilog and terminates the application                            |
| clear            | Resets the active Conversation object in ConversationManager              |
| save             | Serializes the current Conversation to a JSON file via PersistenceService |
| load             | Deserializes a saved JSON file and restores it as the active Conversation |
| stats            | Displays sentiment distribution across all stored messages                |
| search [keyword] | Returns all messages whose content contains the specified keyword         |

#### **4.4 Program.cs, Entry Point and DI Configuration**

Program.cs uses the .NET Generic Host (IHost) to configure and launch the application. The host setup performs several critical operations before ConsoleUI.Run() is called:

- Configures appsettings.json as the primary configuration source, with environment variables as an override layer
- Binds the OpenAI API key from the OpenAI configuration section into an OpenAIOptions object, then checks for an OPENAI\_API\_KEY environment variable override using PostConfigure
- Registers all three services against their interfaces in the DI container
- Configures Serilog as the logging provider, writing to both the console and a rolling daily log file at logs/chatai-.log

By using the Generic Host rather than a simple static Main method, the application gains access to the full .NET dependency injection, configuration, and logging infrastructure with minimal boilerplate.

## **5. Data Flow**

The following describes the complete data flow for a single user message, from console input to displayed response:

| Step                | What happens                                                                                                                                                                                                                                 |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Input            | User types a message and presses Enter. ConsoleUI reads the line.                                                                                                                                                                            |
| 2. Command check    | ConsoleUI checks whether the input matches a command keyword. If yes, the command handler runs and the cycle restarts.                                                                                                                       |
| 3. Storage          | ConsoleUI passes the message to <code>IConversationManager.AddMessage()</code> . The <code>ConversationManager</code> creates a <code>Message</code> object with timestamp and sentiment, then appends it to the <code>Conversation</code> . |
| 4. Context assembly | ConsoleUI calls <code>IConversationManager.GetRecentMessages()</code> to retrieve the last ten messages as context.                                                                                                                          |
| 5. API request      | ConsoleUI calls <code>IOpenAIService.SendAsync(message, context)</code> . The <code>OpenAIService</code> builds the request payload, sends it via <code>HttpClient</code> with Polly retry wrapping, and awaits the response.                |
| 6. Response storage | The API response is returned to ConsoleUI, which passes it to <code>IConversationManager.AddMessage()</code> as an AI-role message.                                                                                                          |
| 7. Display          | ConsoleUI prints the response to the console and loops back to Step 1.                                                                                                                                                                       |

## **6. Implementation Details**

### **6.1 Dependency Injection with .NET Generic Host**

ChatAI uses `Microsoft.Extensions.DependencyInjection` through the `.NET Generic Host` to manage all service lifetimes and dependencies. Services are registered in the `ConfigureServices` callback in `Program.cs`. `ISentimentService` and `IConversationManager` are registered as singletons because they maintain state across the entire session. `IOpenAIService` is registered via

AddHttpClient, which manages the HttpClient lifecycle through IHttpClientFactory to prevent socket exhaustion.

No class in the system constructs its own dependencies using new. Every dependency is declared through a constructor parameter and resolved by the DI container at startup. This makes the dependency graph explicit, testable, and configurable without code changes.

## **6.2 Asynchronous Programming**

All API communication in ChatAI is implemented with C#'s async and await keywords. The OpenAIService.SendAsync() method is awaited by ConsoleUI, and the main application loop runs on an async Task. This ensures the application thread is never blocked waiting for a network response. Without async/await, the console would freeze on every API call until the response arrived, which could take several seconds under normal conditions.

## **6.3 Resilience with Polly**

The OpenAIService wraps all API requests in a Polly retry policy configured with exponential backoff. If a request fails due to a transient HTTP error (5xx), network timeout, or rate limit response (429), Polly automatically retries the request after a delay that doubles on each attempt. This behavior is transparent to the ConsoleUI, from the caller's perspective, the send method either returns a response or throws after the maximum retry count is exhausted. Without this pattern, any transient API failure would surface as an unhandled exception or empty response with no opportunity for recovery.

## **6.4 Structured Logging with Serilog**

ChatAI uses Serilog for all runtime logging. Program.cs configures two log sinks: a console sink for real-time diagnostic output and a rolling file sink that writes to logs/chatai-[date].log with daily rotation. Log entries are written at structured severity levels (Information, Warning, Error, Fatal), which makes it easy to filter logs by severity when diagnosing issues. The Log.Fatal call in the catch block of Program.cs ensures that any uncaught exception that terminates the application is captured with its full stack trace before Serilog flushes and closes.

## **6.5 Configuration Management**

API credentials and application settings are managed through appsettings.json, which is loaded by the Generic Host at startup. The OpenAI API key is bound from the OpenAI configuration section into an OpenAIOptions object using the Options pattern. A PostConfigure callback checks for an OPENAI\_API\_KEY environment variable and overrides the file-based value if one is present. This design allows the application to run in different environments, local development using the settings file and production or CI using environment variables, without code changes.

## **6.6 JSON Persistence**

The PersistenceService uses System.Text.Json to serialize and deserialize the Conversation object. The serialized output includes all Message fields: content, role, timestamp, and sentiment classification. JSON was chosen over binary serialization formats because the output is human-readable, making it easy to inspect saved conversations in a text editor, debug persistence issues, or transfer conversation files between machines without compatibility concerns.

## **7. Testing Strategy**

The Tests directory contains a suite of unit tests built on the .NET test framework. Tests are written to verify the behavior of individual services in isolation, without requiring a live API connection or a running console session. The testing strategy covers three primary areas:

### **7.1 Message Storage and Retrieval**

Tests verify that the ConversationManager correctly appends messages, maintains insertion order, enforces the rolling context window size, and returns the correct subset of messages when GetRecentMessages() is called with varying counts.

### **7.2 Sentiment Classification**

Tests verify that the SentimentService returns the expected classification for a representative set of positive, negative, and neutral input strings. Edge cases such as empty strings and mixed-tone sentences are included to confirm that the classifier handles boundary conditions without throwing exceptions.

### **7.3 Persistence Operations**

Tests verify that a Conversation object serialized by the PersistenceService can be deserialized back into an equivalent object with no data loss. Tests also confirm that loading a non-existent file produces an appropriate error response rather than an unhandled exception.

Unit testing was integrated into the development process rather than added after the fact. Writing tests for each service before integrating them into the full application allowed the team to confirm individual component behavior early and reduced the number of integration-level bugs encountered during final assembly.

## **8. System Requirements**

| Requirement        | Details                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------|
| Runtime            | .NET 6 or later                                                                             |
| API Access         | Valid OpenAI API key (GPT-3.5 or GPT-4 compatible endpoint)                                 |
| Connectivity       | Active internet connection required for all API calls                                       |
| NuGet Dependencies | Polly, Serilog, Serilog.Sinks.File, Microsoft.Extensions.Hosting, Microsoft.Extensions.Http |
| Configuration      | API key set in appsettings.json OpenAI section or OPENAI_API_KEY environment variable       |
| Storage            | Write access to a local directory for JSON conversation files and log output                |

## **9. Limitations and Future Work**

### **9.1 Current Limitations**

- The console interface limits accessibility and user experience compared to a graphical application
- Sentiment classification uses keyword matching rather than a trained model, reducing accuracy on complex, ironic, or mixed-tone input
- The application requires an active internet connection, offline fallback is not supported
- Only one user and one active conversation session are supported at a time
- The rolling context window is fixed at ten messages, which may be insufficient for very long or complex sessions

### **9.2 Future Improvements**

- Graphical user interface, a WPF or MAUI front end would make the application accessible to a broader audience
- ML-based sentiment analysis, replacing the keyword classifier with a trained model (e.g., via ML.NET) would significantly improve classification accuracy
- Cloud persistence, storing conversations in Azure Blob Storage or a database would enable cross-device sync and multi-user support
- Streaming responses, implementing the OpenAI streaming API would allow responses to render token-by-token rather than waiting for the full response
- Configurable context window, exposing the history window size as a setting would allow users to tune the depth of conversational context
- Voice I/O, integrating speech recognition and text-to-speech would enable audio-based interaction

## **10. Conclusion**

ChatAI demonstrates that building a reliable conversational AI application requires more than connecting to an API. Every component of the system, the Generic Host configuration in Program.cs, the interface-based service design, the Polly retry policy in OpenAIService, the rolling context window in ConversationManager, and the structured logging through Serilog, reflects a deliberate engineering decision aimed at producing software that is not just functional but maintainable, testable, and resilient.

The project gave the team hands-on experience with the full stack of .NET application development: dependency injection, asynchronous programming, HTTP client management,



configuration binding, structured logging, JSON serialization, and unit testing. Each of these topics reinforced in practice what is often only described in theory.

The codebase at [github.com/davidbah2703/ChatAi](https://github.com/davidbah2703/ChatAi) represents a working foundation from which more advanced capabilities streaming responses, a graphical interface, cloud storage, and multi-user support can be developed in future iterations.

## **References**

Microsoft Corporation. (2022). .NET Generic Host. Microsoft Learn.  
<https://learn.microsoft.com/en-us/dotnet/core/extensions/generic-host>

OpenAI. (2023). OpenAI API reference. <https://platform.openai.com/docs/api-reference>

App-vNext. (2023). Polly: .NET resilience and transient-fault-handling library.  
<https://github.com/App-vNext/Polly>

Serilog Contributors. (2023). Serilog documentation. <https://serilog.net>

Microsoft Corporation. (2022). Dependency injection in .NET. Microsoft Learn.  
<https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>